

AI-PM 平台使用指南

AI 驱动的项目管理与绩效评估平台 · v1.1 · 2026-05

平台介绍

什么是 AI-PM

AI-PM 是一个 AI 驱动的项目管理平台，深度集成 Jira 数据，为研发团队提供 Sprint 可视化管理、智能风险预警、自动化报告生成、量化绩效评估和发版管理。区别于传统 PM 工具，AI-PM 通过 AI 引擎自动分析数据、识别风险、生成洞察，帮助团队做出更好的决策。

平台优势

优势	说明
零配置接入	直接连接 Jira，无需手动导入数据，实时同步 Sprint/Issue 状态
AI 智能分析	每个页面内置 AI 分析，一键生成项目健康度评估、风险提示和行动建议
量化绩效	基于 SPACE + DORA 模型，五维度客观评估开发者绩效，数据驱动而非主观打分
自动化报告	日报/周报/Sprint 复盘一键生成，支持企业微信推送
多视角看板	PM 和 DEV 角色差异化视图，每个人看到最相关的信息
风险前置	AI 自动识别进度风险、资源过载、范围蔓延，提前预警而非事后补救
Release Notes	自动聚合 Sprint 已完成任务，按分类导出发版说明

参考模型与平台

领域	参考来源	我们的应用
项目管理	Jira Software、Linear、Notion Projects	Sprint 看板、Backlog 管理、时间线视图

团队洞察	Jellyfish、Pluralsight Flow、Waydev	开发者负载可视化、团队热力图、AI 洞察
绩效模型	SPACE 框架（GitHub/微软研究）+ DORA 指标（Google DevOps）	五维度绩效评估（吞吐量/效率/质量/影响力/协作）
AI 能力	GitHub Copilot Workspace、Cursor	AI 分析面板、智能规划、变更摘要生成
通知协作	Slack、企业微信	风险推送、报告推送、AI 助手对话

SPACE 框架由 GitHub 和微软研究院提出，衡量开发者生产力的五个维度：Satisfaction（满意度）、Performance（绩效）、Activity（活动量）、Communication（协作）、Efficiency（效率）。

DORA 指标由 Google DevOps Research 提出，包括部署频率、变更前置时间、变更失败率、恢复时间。我们将这两个框架结合，形成适合内部团队的量化评估体系。

功能模块详解

Dashboard（项目驾驶舱）

视图	内容	适用角色
全局视图	Sprint 完成率、任务状态分布（可点击查看明细）、风险预警数、团队负载柱状图、燃尽图、交付预测	PM / DEV
个人视图	我的待办/进行中/已完成任务列表，个人进度统计	DEV
部门绩效	五维度雷达图、团队成员排名、绩效分布、任务明细	PM
Release Notes	当前 Sprint 发版说明，按分类展示，支持导出 HTML	PM
AI 决策	完成率评估、风险提示、AI 行动建议	PM

Sprint 管理

Tab	功能详情
-----	------

执行看板	5 列状态（待办→进行中→评审中→测试中→已完成），支持按优先级/负责人/关键词筛选，点击 Ticket ID 跳转 Jira
资源视图	按 Developer 字段分组展示开发者卡片，含负载指示器（过载/均衡/空闲）、任务列表、团队统计栏（点击展开明细）
变更检测	AI 自动检测优先级变更、新增需求、范围变更、状态回退；范围蔓延超 20% 自动告警；生成变更影响摘要
AI 规划	从 Backlog 中 AI 评分推荐候选任务，支持勾选后一键复制规划清单

需求管理

管理项目近一年所有需求，支持列表/看板两种视图。顶部状态统计栏可快速筛选。所有 Ticket ID 可点击跳转 Jira。内置 AI 需求健康度分析。

风险与协作

风险看板按状态分列（已识别→评估中→应对中→已关闭）。自动识别：未分配高优任务、超工时任务、长时间无更新任务、跨项目协作依赖。支持一键企微推送风险通知。

报告中心

自动生成日报（当日进展）、周报（工作量+风险汇总+下周计划）、Sprint 复盘（完成率+团队贡献+改进建议）。支持一键推送企业微信。AI 自动生成进展摘要。

项目路线图

水平时间轴展示里程碑和关键节点。4 个内置模板。支持从 Jira Fix Versions 同步里程碑。AI 路线图健康分析。

AI 助手 (Mini Coco)

右下角  按钮打开 AI 对话框。可直接查询 Jira 数据（"DTS 当前 Sprint 完成率？"）、搜索任务、生成报告。对话中的链接和 Ticket ID 可直接点击打开。

设置

Jira 连接配置、通知设置、**排除人员管理**（已离职人员不参与绩效计算和资源展示）。

绩效评估体系

评估模型

基于**SPACE 框架**（GitHub/微软）和 **DORA 指标**（Google DevOps Research），结合 Jira 数据自动计算，形成五维度量化评估：

绩效分 = 吞吐量 × 20% + 效率 × 25% + 质量 × 25% + 影响力 × 15% + 协作 × 15%

绩效等级

- 80-100 优秀
- 60-79 良好
- 40-59 一般
- 0-39 需改进

等级由分数自然产生，不做强制分布。

Jira 优先级定义

优先级直接影响**影响力维度**。P0/P1 属于"高优"，完成高优任务是影响力得分的核心来源。

等级	定义	响应要求	绩效贡献
P0	阻塞生产运营，需立即叫醒开发修复	立即，当天修复	✅ 高优（影响力+60%权重）
P1	紧急问题，无 workaround，当天修复	当天修复	✅ 高优（影响力+60%权重）
P2	严重问题，Ready 后立即发布	尽快修复	普通
P3	下个 Sprint 完成，需设 Due Date	按排期	普通
P4	放入下一个可用 Sprint	按排期	普通

维度一：吞吐量（权重 20%）

衡量什么：Sprint 内完成的工作总量（数量 × 复杂度）。

计算逻辑：完成任务数 × 复杂度因子（1.0~5.0），在团队中算百分位排名。单人团队直接满分。

复杂度因子加分规则

条件	加分	逻辑
子任务 >3 / >6	+0.5 / +1.0	任务拆分多说明复杂（互斥取高）
关联 issue >2 / >5	+0.3 / +0.6	跨模块依赖多（互斥取高）

评论 >5 / >10	+0.3 / +0.6	讨论多说明有挑战（互斥取高）
跨 Sprint N 次	+0.3 × N	大型持续性任务

如何提升：多完成任务 + 接复杂任务 + 确保状态移到 Done。复杂度因子 5.0 的大任务相当于 5 个简单任务。

维度二：效率（权重 25%）

衡量什么：完成速度和按时交付能力。

计算逻辑：

- **Cycle Time 分量 (50%)** = max(0, 100 - 平均天数 × 5)。4 天=80 分，10 天=50 分，20 天=0 分。
- **Delivery Rate 分量 (50%)** = Sprint 结束前完成数 / 总分配数 × 100

核心要点：及时更新 Jira 状态是效率分的命脉。In Progress → Done 的时间差就是 Cycle Time。不更新状态会导致系统用 createdAt→updatedAt 计算，通常非常长。

如何提升：拿到任务立刻移 In Progress → 做完立刻移 Done → 拆分大任务控制在 10 天内 → Sprint 内完成承诺。

维度三：质量（权重 25%）

衡量什么：一次做对的能力，是否引入缺陷。

计算逻辑：((1 - 返工率) × 100 + (1 - Bug引入率) × 100) / 2

- **返工率** = 被 Reopen 的任务数 / 已完成任务数
- **Bug 引入率** = 关联了 Bug 类型 issue 的已完成任务数 / 已完成任务数
- Bug 数据不可用时，100% 权重归入返工率

如何提升：开发前确认需求 → 充分自测 → Code Review 自查 → 减少 Reopen。**返工率 0 + Bug 引入率 0 = 满分 100。**

维度四：影响力（权重 15%）

衡量什么：你做的任务对业务的重要程度。

计算逻辑：高优完成比 × 100 × 60% + 阻塞解决速度 × 40%

- **高优** = P0 (Highest) / P1 (High) 任务
- **阻塞解决速度** = max(0, 100 - 有关联 issue 任务的平均 Cycle Time × 5)
- 无阻塞任务时取中性值 50

注意：只做 P3/P4 时，高优完成比=0，影响力最多只有 20 分。主动承担 P0/P1 紧急任务是影响力的决定性因素。

维度五：协作（权重 15%）

衡量什么：你对团队的帮助和跨职能参与。

计算逻辑：

- **评论分量 (50%)** = min(100, 在他人任务上的评论数 × 10)。10 条即满分。
- **任务参与度 (50%)** = 你参与过评论的他人任务数 / 他人任务总数 × 100

最易提分维度：在 10 个不同同事的 Jira ticket 上留有价值的评论（Code Review 建议、技术讨论、问题反馈）即可拿到评论满分。覆盖面比深度更重要。

常见问题

问题	原因与解决
效率分为 0	Jira 状态没及时更新。开始做 → 移 In Progress，完成 → 移 Done。不更新会导致 Cycle Time 按创建到最后更新算（通常几十天）。
吞吐量低但完成了很多	吞吐量是团队内相对排名。接复杂任务（复杂度因子 3.0~5.0）比做很多简单任务（1.0）更有效。
影响力很低	只做了 P2/P3/P4 任务。影响力 60% 来自高优（P0/P1）完成比。需要主动承担紧急重要任务。
协作分低	没有在别人任务上留评论。10 条跨团队评论即可拿满评论分量。多参与 Code Review。
质量不满分	有被 Reopen 或任务关联了 Bug。开发前确认需求，充分自测后再提交。
某人已离职但还显示	进入 设置 → 排除人员 → 添加该人姓名（与 Jira 一致）即可隐藏。

快速提分行动清单

优先级	行动	影响维度	预期提升
☆☆☆	及时更新 Jira 状态（开始→In Progress，完成→Done）	效率	+20~30
☆☆☆	Sprint 内完成承诺的任务，不拖到下个 Sprint	效率 + 吞吐量	+15~25

☆☆☆	一次做对，自测充分，减少 Reopen	质量	+10~25
☆☆	在 10 个同事的任务上留有价值评论	协作	+50 (满分)
☆☆	主动接 P0/P1 高优紧急任务	影响力	+20~40
☆☆	接复杂任务 (多子任务/多关联/多讨论)	吞吐量	+10~20
☆	拆分大任务，控制 Cycle Time < 10 天	效率	+10~15

AI-PM Platform · v1.1 · 2026-05 · <https://ai-pm-platform.item.pub>
绩效模型参考：SPACE Framework (GitHub/Microsoft Research) + DORA Metrics (Google DevOps Research)